

A) la mémoire

1) Qu'est-ce que la mémoire ?

La mémoire est l'espace de stockage où un programme stocke et manipule des données pendant son exécution. C'est comme une vaste grange divisée en cases (octets) numérotées (adresses) où chaque case peut contenir une valeur.

Caractéristiques principales : La mémoire peut être :

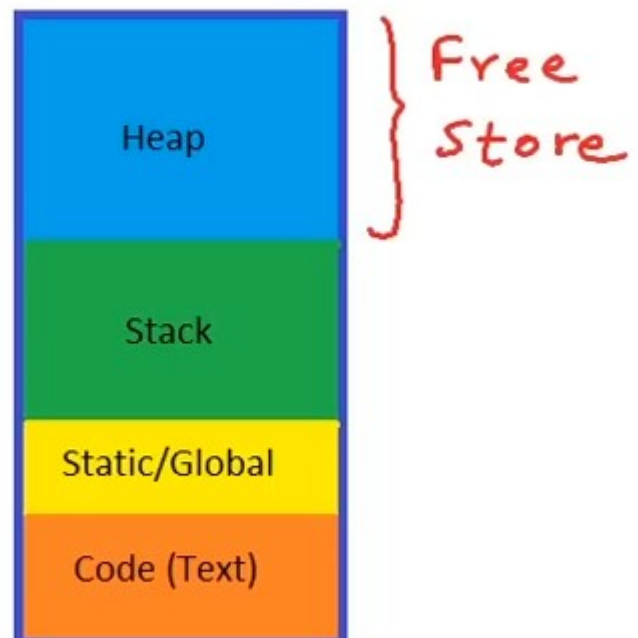
- **Volatile (RAM)** : Perd son contenu quand le programme se termine ou la machine s'éteint
- **Persistante ou non volatile(disque dur, SSD)** : conserve son contenu même après que la machine se soit éteint
- **Adressable** : Chaque octet a une adresse unique
- **Hierarchique** : Cache → RAM → Disque dur

2) Stack et Heap

En programmation, on s'intéresse surtout à la mémoire volatile(RAM) qui est utilisée pour exécuter les programmes. Cette mémoire au regard de l'exécution d'un programme peut être subdivisée en 4 parties :

- **Code Text**: toutes les séquences d'instructions nécessaires pour l'exécution du programme sont stockées dans cette partie
- **Static/Global** : toutes les variables (Variable globale, constante) qui doivent être modifiées n'importe où dans le programme sont stockées ici
- **Stack (pile)** : C'est une région de mémoire qui suit le modèle LIFO (Last In, First Out). Elle est utilisée pour les variables locales et les appels de fonctions. La pile est gérée automatiquement par le compilateur. L'allocation et la libération sont très rapides. La taille de la stack est limitée.
- **Heap** : C'est une région de mémoire qui permet une allocation dynamique. Le heap est géré manuellement par le programmeur (en C/C++) ou par le garbage collector (dans d'autres langages). La taille du heap est plus grande et l'allocation/libération est plus lente.

Application's
memory



B) Pointeur et Mémoire

1) Pointeur

Un pointeur est une variable qui contient l'adresse de l'espace mémoire(dans la RAM) dans lequel est stocker une autre variable. Il permet de manipuler indirectement la valeur de la variable associer à cette espace mémoire .

```
int variable = 42;  
int* pointeur = &variable; // pointeur contient l'adresse de variable
```

Relation avec la mémoire :

- Permet d'accéder directement à n'importe quelle adresse mémoire
- Permet l'allocation dynamique
- Essentiel pour les structures de données complexes

2) **Référence** : C'est la variable elle-même, accessible sous un nom différent.

Une référence est un alias, c'est-à-dire un autre nom pour une variable existante. Une référence doit être initialisée lors de sa déclaration et ne peut pas être modifiée pour référencer une autre variable.

```
int original = 100;  
int& ref = original; // ref est un alias de original
```

3) Différence entre pointeur et référence

- Un pointeur peut être réaffecté pour pointer vers différentes adresses, tandis qu'une référence ne peut pas être réaffectée après son initialisation.
- Un pointeur peut être nul (nullptr), alors qu'une référence doit toujours être liée à une variable valide.
- On peut avoir des pointeurs de pointeurs, mais pas des références de références.
- La syntaxe : avec les pointeurs, on utilise l'opérateur * pour déréférencer, et avec les références, on utilise directement le nom.

5) Passage de paramètres

- **Par valeur(Copie)** : Une copie de la valeur de l'argument est passée à la fonction. Les modifications dans la fonction n'affectent pas l'original.

```
void modifier(int copie) {  
    copie = 100; // Ne modifie pas l'original  
}  
  
int main() {  
    int x = 5;  
    modifier(x); // x reste à 5  
}
```

- **Par pointeur (adresse)** : L'adresse de la variable est passée en paramètre à la fonction. La fonction peut donc modifier la valeur de la variable originale en déréférençant le pointeur.

```
void modifier(int* ptr) {  
    *ptr = 100; // Modifie la variable originale  
}  
  
int main() {  
    int x = 5;  
    modifier(&x); // x devient 100  
}
```

- **Par référence(Alias)**: La fonction reçoit une référence à la variable originale. Les modifications affectent l'original.

```
void modifier(int& ref) {  
    ref = 100; // Modifie la variable originale  
}  
  
int main() {  
    int x = 5;  
    modifier(x); // x devient 100  
}
```

c) Partie Pratique

1) Manipulation de pointeurs

```
#include <iostream>

void demonstrationPointeurs() {
    // Création d'un pointeur sur int
    int* pointeurSurInt;

    // Attribution d'une valeur à la variable pointée
    int variable = 42;
    pointeurSurInt = &variable; // pointeurSurInt contient l'adresse de variable

    // Accéder à la valeur pointée (DÉRÉFÉRENCEMENT)
    std::cout << "Valeur pointée: " << *pointeurSurInt << std::endl;

    // Attribuer une valeur à travers le pointeur
    *pointeurSurInt = 100; // variable devient 100

    std::cout << "Nouvelle valeur de variable: " << variable << std::endl;

    // Qu'est-ce que déréférencer ?
    // DÉRÉFÉRENCER = Accéder à la VALEUR située à l'adresse mémoire stockée dans le pointeur
}
}
```

Notations * &

* (Étoile)

- **Déclaration** : `int* ptr` → déclare un pointeur
- **Déréférencement** : `*ptr` → accède à la valeur pointée

& (Esperluette)

- **Adresse** : `&variable` → donne l'adresse de la variable
- **Référence** : `int& ref` → déclare une référence

2) Structure Étudiant

Version C (malloc/free)

```
#include <stdlib.h>
#include <string.h>

struct Etudiant {
    ... int matricule;
    ... int age;
    ... char nom[50];
};

// Fonction avec malloc/free
struct Etudiant* creerEtudiantMalloc(int mat, int age, const char* nom) {
    ... // Allocation mémoire
    ... struct Etudiant* etud = (struct Etudiant*) malloc(sizeof(struct Etudiant));
    ...
    ... if (etud != NULL) {
    ...     ... (*etud).age = mat;
    ...     ... (*etud).age = age;
    ...     ... strncpy((*etud).nom, nom, 49);
    ...     ... (*etud).nom[49] = '\0'; // Sécurité
    ... }
    ...
    ... return etud;
}

void supprimerEtudiantMalloc(struct Etudiant* etud) {
    ... free(etud); // Libération mémoire
}
```

Version C++ (new/delete)

```
#include <iostream>
#include <cstring>

struct Etudiant {
    int matricule;
    int age;
    char nom[50];
};

// Fonction avec new/delete
Etudiant* creerEtudiantNew(int mat, int age, const char* nom) {
    // Allocation mémoire + construction
    Etudiant* etud = new Etudiant;

    etud->matricule = mat;
    etud->age = age;
    strncpy(etud->nom, nom, 49);
    etud->nom[49] = '\0';

    return etud;
}

void supprimerEtudiantNew(Etudiant* etud) {
    delete etud; // Destruction + libération
}
```

Différences entre malloc/new et free/delete

- **malloc** alloue un bloc de mémoire de taille spécifiée, mais ne construit pas l'objet (n'appelle pas le constructeur).

new alloue la mémoire et appelle le constructeur de l'objet.

- **new** et **delete** sont des opérateurs en C++, alors que **malloc** et **free** sont des fonctions de la bibliothèque C

- **new** et **delete** sont type-safe, alors que **malloc** et **free** ne le sont pas.

3) Arithmétique des Pointeurs

```
void arithmetiquePointeurs() {
    int tableau[5] = {10, 20, 30, 40, 50};
    int* ptr = tableau; // ptr pointe vers tableau[0]

    std::cout << "Valeur actuelle: " << *ptr << std::endl; // 10

    // Incrémentation du pointeur
    ptr++; // Se déplace de sizeof(int) bytes
    std::cout << "Après ptr++: " << *ptr << std::endl; // 20

    // Addition avec un entier
    ptr = ptr + 2;
    std::cout << "Après ptr + 2: " << *ptr << std::endl; // 40

    // Soustraction entre pointeurs
    int* debut = tableau;
    int* fin = tableau + 4;
    std::cout << "Distance: " << (fin - debut) << std::endl; // 4

    // Parcours de tableau avec pointeurs
    for (int* p = tableau; p < tableau + 5; p++) {
        std::cout << *p << " ";
    }
    // Output: 10 20 30 40 50
}
```

Règles de l'arithmétique des pointeurs :

- **ptr + n** : Avance de $n * \text{sizeof}(\text{type})$ bytes
- **ptr - n** : Recule de $n * \text{sizeof}(\text{type})$ bytes
- **ptr1 - ptr2** : Donne le nombre d'éléments entre les deux pointeurs
- **Comparaisons** : ==, !=, <, > possibles entre pointeurs du même tableau

Attention aux pièges :

```
void piegesArithmetique() {  
    int* ptr = new int[10];  
  
    // ⚠ DANGER : Dépassement de tableau  
    int* danger = ptr + 15; // Comportement indéfini  
  
    // ⚠ DANGER : Pointeurs de types différents  
    char* charPtr = (char*)ptr;  
    charPtr++; // OK mais attention aux alignements  
  
    // ✅ Sécurisé : Restez dans les limites  
    for (int i = 0; i < 10; i++) {  
        *(ptr + i) = i; // Équivalent à ptr[i]  
    }  
  
    delete[] ptr;  
}
```